

FREE GUIDE • 2026

The AI Coding Playbook.

How to work with AI coding agents like a team,
not a genie. Practical habits, in plain words.

By Dmytro Shved

What's inside

AI coding agents are fast. Left alone, they are also fast at building the wrong thing, guessing at bugs and saying "done" when it isn't. These are the habits that fix that.

01 The one idea

02 Keep the rules short

03 Match process to task

04 Understand before building

05 Write the task down

06 Never accept "done"

07 Root cause, not loops

08 Look for the twins

09 Testing

10 Reviews must prove it

11 Write decisions down

12 Red flags

Works with any AI coding agent, and even with a single chat window.

01

The one idea

You are the lead. The agent is fast hands with no memory.

The agent can't see your conversations, the design you looked at, or the decision you made an hour ago. It knows only what you wrote down and what it can find in the code.

So the job changes. Less typing, more deciding, explaining and checking.

Three habits matter most:

- 1 Understand** the task before anything is built.
- 2 Fix at the cause**, not where the problem shows.
- 3 Check before you ship**, not after.

Keep the rules short

Most agents read a rules file at the start of every session. It's tempting to put everything in it.

Don't. A rules file that keeps growing stops being followed. The important lines drown in the rest.

- Keep the always-on file short: how the agent should work with you and the few rules that apply everywhere.
- Move detail closer to where it's needed: notes next to the code they're about, or instructions the agent loads only for that kind of job.
- When a rule keeps being broken, turn it into a check that fails, not a longer paragraph.

Match the process to the task

TINY CHANGE

One line, a label, a setting. Just do it.

QUESTION

Ask it. No process needed.

BUG

Find the cause first. Never skip this, even for a one-line fix.

FEATURE

Understand it, write the task down, check the result.

Don't over-process small work. If writing the instructions takes longer than the change, just make the change.

Understand before building

Most wasted AI work starts with a task nobody fully understood. Before the first edit, answer:

- 1 **What exactly is being asked?** One sentence. Then the smallest version of it.
- 2 **Does something already do this?** Search for the behavior, not the name.
- 3 **What happens at the edges?** Empty states, errors, permissions, old data, the other platform.
- 4 **What does "done" look like?** Something a person can see and check.

Ask people only the real product questions, each with your suggestion. Technical choices: decide and move on.

05

Write the task down

Same agent, same model, very different results. The difference is the task.

VAGUE

"Add an export button to the reports page."

CLEAR

The goal. Which files. What not to touch. How you'll know it works. What to do if unsure.

A clear task is cheaper **and** more accurate. A vague one makes the agent explore, guess and "improve" things nobody asked for.

Tell the agent: if unsure, stop and ask. An early question beats a confident guess.

Never accept “done”

An agent saying "done" is a claim, not proof.

- **Read the real changes**, not the summary.
- **Check what it removed**. This is where agents fail most often.
- **Check what it added that nobody asked for**, and take it out.
- **Run the check yourself** for anything that could ship wrong.

Don't correct a running agent. A message mid-task gives you two half-plans. Let it finish or stop it, fix the task, start again.

Root cause, not loops

Fix where the problem **starts**, not where it shows. A patch makes one case pass and leaves the cause alive. Then the patches start fighting each other.

Warning signs of a patch:

- a special case for this one input, user or screen
- a delay, a magic value, or an error that gets swallowed
- the same correction added in a second place
- a test changed only so it passes

The same problem came back? Then the last fix was a patch. Stop and find the real cause from scratch.

Look for the twins

Where a bug was reported is where it was **found**, not where it **is**.

Mistakes get copied. Before calling a fix done, look for the same mistake in other places: other screens, the other platform, emails and exports, background jobs, scripts.

Then make it hard to come back: one shared place for the rule, a database constraint, or a test you've watched fail.

"Found nothing else" is a claim. A quick test beats an hour of reading.

Testing

- **Reproduce first.** See the bug happen before touching anything.
- **Watch every new test fail** on the old code first. A test you never saw fail proves nothing.
- **Run it, don't just read it.** Passing types are not proof. Use the real screen or endpoint once.
- **Test only what you touched** while working. Everything before you ship.

Your test data should try to break your fix: old formats, empty fields, two where you expect one, records that should be filtered out.

Reviews must prove it

AI reviewers write convincing text. Many findings sound right and are wrong, and each one costs you time.

A finding is a demonstration, not an argument.

Ask the reviewer to show the problem: a failing test, or the exact steps to reproduce it. No demonstration, no finding.

Some things still need a human to think: design problems, security, race conditions. But for everyday bugs, this cuts the noise a lot.

Write decisions down

The agent forgets everything when a session ends. Anything not written down is lost.

- **Keep a decisions log.** What was decided and why, newest first. Check it before changing direction.
- **End sessions with a hand-off note:** where things stand and what's next.
- **When you correct the agent,** write the correction down so you never make it twice.

Red flags: stop if you see one

- Code written before anyone understood the task.
- A fix with no reproduction ("it's probably...").
- The same problem fixed twice.
- A special case added for one input.
- A test never seen failing, or edited just to pass.
- Accepting an agent's summary without reading the changes.
- A correction sent to a running agent.
- Features nobody asked for.
- A review "finding" with no proof.
- A decision that lives only in chat.

Start tomorrow

Three habits that change the most:

- 1 **Write the task down** before the agent starts: goal, limits, how you'll check it.
- 2 **Read every change** for what was removed and what was added unasked.
- 3 **Same problem twice? Stop.** Find the real cause before the next fix.

If this guide was useful, pass it on to one developer who would use it.